Comparing Black-Box, White-Box, and Usage-Based Testing Techniques on Perch, a Full-Stack Housing and Social Platform for Interns: Progress Report

Jayden Manyrath and Koen Gennis

CS 7314/5314, Fall 2026 | Prof. Jeff Tian | Project Progress Report | October 29, 2026

1. Summary of the Proposal

We proposed to apply four formal testing techniques to Perch, our Next.js and Supabase web app that helps interns find sublets, friends, and events in a new city. We use finite state machine testing (FSM/BBT) on the booking and onboarding workflows, partition testing (PT/BBT) on the affordability and housing-verdict rules, control flow testing (CFT/WBT) on the branch-heavy rule and fallback code, and Markov chain operational profile testing (Markov-OP/UBST) to exercise the app according to a projected usage profile. The repository's existing Vitest suite is the baseline. We compare the techniques on defects found, cost, coverage, and overlap, and we track failures across test rounds. By this report we planned to have the FSM, PT, and CFT suites built and executed, defects fixed and retested, and the Markov chain built.

2. Changes in Scope

We made a few adjustments after the code walkthrough. The booking states in the code use slightly different names than our proposal ("requested" instead of pending, "booked" instead of confirmed), and the listing has its own status (available, taken, or stale), so our FSM states pair the booking status with the listing status. We also found that when no live keys are set, which is the app's default demo mode, every booking action runs against built-in fixture data instead of the server. Since that is a second implementation of the same rules, we ran the FSM model against both the live API code and the fixture code. For FSM, PT, and CFT we ran against an in-memory copy of the database instead of the live Supabase instance so that every run is repeatable. The live environment is saved for the usage-based runs. Finally, the volunteer pilot slipped about one week. The Markov chain is built and checked, but the pilot data will come in early November.

3. Progress So Far

Baseline. Before writing any new tests we measured the existing suite. It has 499 test cases, all of which pass except 7 that need live keys, and it covers 76.7% of the branches in the modules we are targeting. It never runs the listing-expiry job, the server-side booking transition code, or the booking fallback paths. Because it passes completely on the original code, it caught none of the defects described below.

FSM/BBT (Jayden). The booking model has seven states (listed, requested, approved, booked, declined, cancelled, and stale) and thirteen valid transitions. From it we built test cases that cover every valid transition and eighteen invalid ones, such as approving twice, confirming a taken listing, or a second intern requesting a listing that is already held. We also covered five longer sequences and two cases where two actions happen at the same moment. The onboarding model follows the eight onboarding screens and checks the rule for leaving the review screen. We ran it in a real browser. Across 69 FSM test cases, 11 failed in the first round, traced to seven defects. A subletter could relist a taken listing while it was still booked. Two interns requesting at the same instant gave the loser a server error instead of a clear "already held" response. In fixture mode, booked bookings could be re-approved or declined, and taken or stale listings could be requested again. Onboarding accepted an end date before the start date. Building the model also showed that the cancel step is described in the design but has no route in the app.

PT/BBT (Koen). We split seven inputs into valid and invalid classes and tested each side of every boundary: rent against budget, salary against tax bracket, cost-of-living index, how pay is written in the offer letter, lease dates against internship dates, walking time, and safety flags. All expected values were worked out by hand from the documented tax and rent rules. Every one of the 19 hand-computed tax and rent values matched the code. The 10 failures out of 78 cases came from four defects in the unusual and invalid classes. The most serious was that an offer saying "$45.00 per hour" was read as a $45 yearly salary, which makes every listing fail the budget check. Intern offers are often hourly, so this is a common case. We also found that the

negotiation route ignored a budget the intern set lower than their salary allowed, accepted badly formatted or reversed dates, and showed "Gap of NaN days" on bad lease dates.

CFT/WBT (Koen). We drew control flow graphs for the listing-expiry logic and its scheduled job, the booking transition code, and the code that falls back to fixture data when the server fails. We then wrote 37 test cases to force each branch. The expiry and transition code had no defects, but this raised branch coverage there substantially (the expiry job went from 0% to 88%). The fallback graph exposed the second serious defect. When the server rejected a booking on purpose, for example because the listing was already held, the app treated it like a network outage, fell back to fixture data, and showed the user a fake successful request.

Markov-OP/UBST (Jayden). We built a usage model over nine main screens (onboarding, feed, perches, listing detail, booking, negotiate, map, chirps, and profile) plus start and exit states. The starting probabilities come from the designed user journey and team estimates. We checked that the model is well formed and that simulated sessions match the expected visit counts, and we wrote a generator that draws repeatable test sessions from it. The first 200-session suite averages about five screens per session, with the feed and perches screens visited most.

Results so far. In total we logged 14 defects with ODC type and trigger, and we fixed and retested 12 of them. The two still open are the missing cancel route and fixed stay dates on the negotiate screen, which we found during the walkthrough. Failing test cases dropped from 25 in round one to zero in round two, and none of the existing 492 tests broke. Each technique found the kind of defect its model targets. FSM found the most (eight), all involving the order of events or two users interacting. PT found four, all on input boundaries. CFT found one, but it was one of the two most severe and no black-box test could have seen it. No defect was found by more than one technique, which supports using several. Together the new suites raised branch coverage on the target modules from 76.7% to 83.3%. They run in under two seconds, plus about a minute for the browser tests, so building the models was most of the cost. We are logging hours per technique to compare defects per hour in the final report.

4. Remaining Work

The main remaining work is the usage-based testing and the overall comparison, following the schedule below.

Dates (2026)	Milestone
Oct 30 to Nov 6	Add navigation logging; run the pilot with 10 to 15 volunteers; refine the usage profile
Nov 2 to Nov 13	Run UBST sessions against the live environment over several rounds; estimate reliability; sensitivity check
By Nov 13	Decide on the cancel route and stay-date fixes with instructor feedback; finish the effort log
By Nov 20	TA demo
Nov 21 to Dec 8	Cross-technique analysis; add suites to GitHub Actions; final report (due Dec 8)